

GlobalLogic (A Hitachi Group Company)

BSC-NGT Portfolio — GenAI Consultancy

Exp Intelligence

AI-Assisted Storefront Optimization Pipeline

Technical Architecture, Rationale & Strategic Roadmap

by Aadi Jain

“Turn raw storefront analytics into ranked, data-grounded improvement suggestions — each with a ready-to-paste code patch — reviewed by a product owner and picked up by developers.”

Author: Aadi Jain — Backend & Backlog

Programme: GenAI Consultancy Internship, BSC-NGT

Document type: Technical & strategic report

Scope: Context, Why, How, PM & delivery, Improvements, Future, Deployment

Services covered: Shopify Extractor, Analytics, AI, Review Hub

Date: July 2026

Contents

1	Executive Summary	2
2	Project Context & Organization	2
2.1	Team & stakeholders	3
2.2	The v1 success criterion	3
2.3	Source repositories	3
3	Why — The Problem & Motivation	4
3.1	The two-buyer problem	4
3.2	The problem	4
3.3	The insight	4
3.4	Why a human-in-the-loop design	5
4	How — System Architecture	5
4.1	Architecture at a glance	5
4.2	End-to-end data flow	5
4.3	Service 1 — Shopify Extractor (<code>shopify-pp</code>)	6
4.4	Service 2 — Analytics Service (<code>analytic service</code>)	6
4.5	Service 3 — AI Service (<code>ai service</code>)	7
4.6	Service 4 — Review Hub (<code>review-hub</code>)	7
5	Product Management & Delivery	8
5.1	Agile backlog	8
5.2	Key product decisions (and their rationale)	8
5.3	Competitive & discovery context	9
6	How to Connect All Services	9
6.1	The wiring map	9
6.2	Local development	9
6.3	Exposing local services with Cloudflare quick tunnels	10
7	Deployment — Vercel, Railway, Cloudflare & More	10
7.1	Review Hub on Vercel (or Cloudflare Pages)	10
7.2	Shopify Extractor & AI Service on Railway	11
7.3	Analytics Service via Docker (Railway / Fly.io / any container host)	11
7.4	Cloudflare’s role	11
7.5	Deployment checklist	12

8	How Can We Improve It	12
8.1	Persistence & state	12
8.2	Configuration & security	12
8.3	Reliability & observability	12
8.4	AI quality	13
9	Future Scope	13
9.1	Immediate next steps (open items)	13
9.2	Vision	13
9.3	Pillar 1 — Universal connectors (beyond Shopify)	14
9.4	Pillar 2 — Automatic code application	14
9.5	Pillar 3 — Continuous optimization	15
9.6	Target architecture	15
10	Conclusion	15

1 Executive Summary

Exp Intelligence is a microservice pipeline that closes the loop between *observing* how a Shopify storefront performs and *acting* on it. It automatically pulls a store's own analytics, detects statistically significant problems, uses large language models (LLMs) to translate those problems into prioritized, plain-English improvement suggestions — each carrying a concrete code patch — and presents them in a review dashboard where a Product Owner (PO) approves or rejects them and developers pick up the approved ones.

The system is composed of four independently deployable services:

Service	Folder	Port	Responsibility
Shopify Extractor	<code>shopify-pp</code>	3000	OAuth install on a store; extract ShopifyQL metrics + real theme code; forward to Analytics
Analytics Service	<code>analytic service</code>	4000	Normalize & ingest data; run a Playwright crawler; detect problems statistically
AI Service	<code>ai service</code>	5001	Turn analytics + screenshots into ranked suggestions and code patches via LLMs
Review Hub	<code>review-hub</code>	5173	React dashboard: PO approve/reject, developer code view

Data flows strictly in one direction:

`shopify-pp` → `analytic service` → `ai service` → `review-hub`

The core value proposition

Store owners drown in dashboards but starve for decisions. Exp Intelligence does not add another dashboard — it produces *ranked, data-grounded actions with the code to implement them*, gated by a human review step so that nothing ships without a person approving it.

2 Project Context & Organization

Exp Intelligence (also referred to as the *Experience Intelligence / Analytics-to-UX Recommendation Engine*) was built during a GenAI Consultancy internship engagement within GlobalLogic's **BSC-NGT** (Business Services & Consulting — Next Generation Technologies) portfolio. It targets D2C (direct-to-consumer) Shopify merchants in the fashion, beauty, and lifestyle segment.

2.1 Team & stakeholders

Name	Role	Relevance to the project
Ambuj Prakash	Project Mentor & Lead	Reviews deliverables, overall engagement ownership
Nikhil Gupta	Technical Mentor	Technical architecture & implementation guidance
Shubham Tomar	Mentor	Onboarding, guidance, reviews
Akanksha Rajwanshi	Mentor	Onboarding, guidance, reviews
Samad Khan	Team / BSC-NGT	BSC-NGT Portfolio team
Mansimar	Peer — Backend & Backlog	Co-owned backend (Analytics + AI services) & backlog
Aadi Jain	Peer — Backend & Backlog	Co-owned backend (Analytics + AI services) & backlog
Pranjal Tanwar	Peer — Frontend & Backlog	Co-owned frontend (Admin Portal / Review Hub) & backlog
Rishabh Varshney	Peer — Frontend & Backlog	Co-owned frontend (Admin Portal / Review Hub) & backlog

2.2 The v1 success criterion

The success criterion agreed with the team is deliberately narrow and outcome-based: **one real merchant pilot where at least one AI-generated recommendation is actually implemented on the merchant’s store and its impact measured.** Everything in this report — the architecture, the review gate, the code generation — exists to make that single outcome achievable and repeatable.

2.3 Source repositories

Each service lives in its own GitHub repository (assembled as submodules of the `exp-intelligence` monorepo):

Service	Repository
Shopify Extractor	https://github.com/aadij1905/shopify-extractor
Analytics Service	https://github.com/aadij1905/analytics-service
AI Service	https://github.com/aadij1905/ai-service
Review Hub	https://github.com/aadij1905/review_hub

A note on this document vs. the handover snapshot

Parts of this report draw on the internship *handover* document. The *technical* sections here reflect the **current codebase**: all four backlog epics — including E-004 code generation — are complete and in MVP scope, and suggestion generation supports *multiple* LLM providers with a generate→judge orchestration rather than a single fixed model. Where the handover and the code differ in incidental detail (service ports, the “Admin Portal”/“Review Hub” name, the default model), this report follows the code.

3 Why — The Problem & Motivation

3.1 The two-buyer problem

The engagement was framed around a **two-buyer problem** that shaped most product decisions:

- **The Growth Lead / marketer** feels the conversion (CRO) pain daily but has *no access to the store's codebase* to act on it.
- **The Developer** controls the codebase and can make changes, but has *no visibility* into analytics or CRO priorities.

Neither can close the loop alone. This is why the product deliberately runs two tracks — an insight/prioritization/business-impact experience for the Growth Lead (the PO view), and an implementation-ready experience with code patches for the Developer — and why a human approval step sits between them. The sections below expand the underlying analytics and engineering rationale.

3.2 The problem

Shopify merchants have access to an enormous amount of analytics — sessions, conversion rates, bounce rates, traffic sources, device breakdowns, checkout funnels, Core Web Vitals — but this data is:

- **Descriptive, not prescriptive.** A dashboard tells you the product page bounce rate is 71%. It does not tell you *why*, whether that is statistically meaningful given the traffic, or *what to change in the code* to fix it.
- **Disconnected from implementation.** Even when a merchant knows *what* is wrong, translating “improve the product page CTA” into an actual edit in a Liquid theme file is a developer task that rarely happens.
- **Noisy.** A wild-looking metric over six sessions is almost certainly noise, yet naive tools flag it anyway, eroding trust.

3.3 The insight

The gap is not *more measurement* — it is the translation layer between *measurement* and *implementation*. Three capabilities, chained together, close that gap:

1. **Statistical rigor** to separate real problems from sampling noise (a one-sample *z*-test on proportions, not raw multipliers).
2. **Multimodal reasoning** — combining the *numbers* (ShopifyQL) with what a page *looks like* (crawler screenshots analyzed by a vision model) — because many conversion problems (a CTA below the fold, missing social proof) are invisible in metrics alone.
3. **Code generation grounded in the store's own theme**, so a suggestion arrives as a real patch against the real installed theme, not generic advice.

3.4 Why a human-in-the-loop design

Every suggestion passes through a Product Owner before a developer sees it. This is deliberate: LLM output is probabilistic, and shipping automated code changes to a live storefront without review is unacceptable risk. The PO layer turns the AI from an “autonomous agent” into a *trusted assistant* — the AI proposes, a human disposes, and only then does a developer implement. The roadmap (Section 9) describes how this gate is preserved even as the system moves toward auto-applying patches.

4 How — System Architecture

4.1 Architecture at a glance

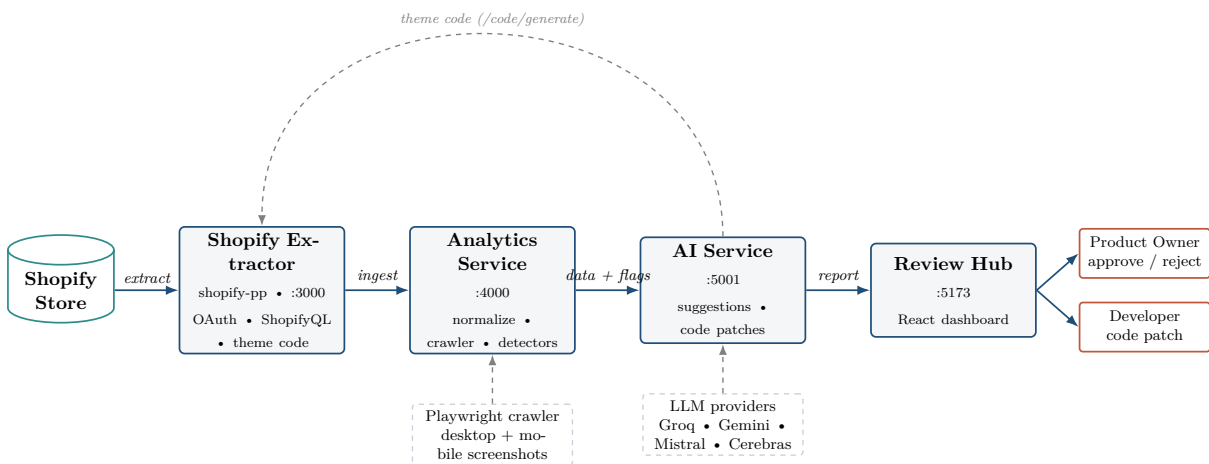


Figure 1: The Exp Intelligence pipeline. Solid arrows are the one-directional data flow; dashed arrows are supporting dependencies (the crawler and LLM providers) and the on-demand theme-code fetch used when generating a patch.

4.2 End-to-end data flow

1. **Install & extract** (*shopify-pp*). A merchant installs the embedded app on their store via Shopify OAuth. The extractor runs 10 ShopifyQL queries (overview, sales, per-page, traffic sources, devices, funnel, checkout conversion, and three Core Web Vitals: LCP, CLS, INP) and can also pull the *real theme code* that renders any given page.
2. **Ingest & normalize** (*analytic service*). Raw query rows are normalized into a uniform shape, then a headless Playwright crawler visits the storefront, takes desktop + mobile screenshots, and enriches page records (e.g. “is the CTA above the fold?”, “is there social proof?”).
3. **Detect** (*analytic service*). Deterministic detectors run a z -test against the site baseline and emit ranked, severity-scored flags (high-bounce page, low-conversion page, poor traffic source, cart abandonment, funnel drop-off, revenue decline, poor Core Web Vitals, CTA below fold).
4. **Generate** (*ai service*). The AI service pulls the normalized data + flags, and — if screenshots exist — feeds them to a vision model in one integrated pass. LLMs draft

suggestions; a deterministic validator grounds every cited number; a revenue forecaster attaches a data-derived impact estimate.

5. **Review** (review-hub). The PO sees suggestions *without* code (problem, plain-English recommendation, effort, impact, confidence) and approves or rejects. Developers see only approved suggestions, each with a ready-to-paste code patch generated on demand against the store's real theme.

4.3 Service 1 — Shopify Extractor (shopify-pp)

Stack: Node.js + Express, better-sqlite3 for persistence.

- **OAuth & security.** Implements the full Shopify OAuth handshake with HMAC verification on both the install redirect and webhooks (raw-body HMAC, mounted before `express.json()` so signatures verify against the exact bytes). Access tokens are encrypted at rest in SQLite using a 32-byte `ENCRYPTION_KEY`.
- **Compliance.** Handles Shopify's mandatory GDPR webhooks (`customers/data_request`, `customers/redact`, `shop/redact`). The app stores no customer PII — only shop domain + access token — so redaction simply deletes the shop record.
- **ShopifyQL extraction.** A single `SESSION_FILTER` constant guarantees every query (`overview`, `pages`, `traffic`, `devices`, `funnel`, `checkout`) counts the *same* session universe, so the cards never disagree.
- **Theme code extraction.** `/api/debug/theme-code` returns the actual Liquid files that render a given page — the raw material the AI service uses to write store-specific patches.

4.4 Service 2 — Analytics Service (analytic service)

Stack: Node.js + Express, Playwright (Chromium), optional AWS S3 for screenshot storage.

- **Normalization.** Converts raw ShopifyQL rows into a stable internal shape; Core Web Vitals are bucketed against Google's own thresholds (LCP good $\leq 2500\text{ms}$, CLS good ≤ 0.1 , INP good $\leq 200\text{ms}$).
- **Crawler.** A reusable headless Chromium session visits each significant page at desktop (1280×800) and mobile (390×844) viewports. It detects the add-to-cart CTA using known theme selectors *plus* a theme-agnostic text heuristic (“add to cart / buy now / ...”), measures whether it is above the fold, and detects social-proof widgets.
- **Statistical detectors.** The heart of the noise-rejection strategy. See the callout below.

Why a z -test instead of a raw multiplier

A page bouncing at $2\times$ the site average over 6 sessions is probably noise; the same rate over 6,000 sessions is a real signal. A fixed “minimum sessions” floor cannot tell these apart across store sizes. The one-sample z -test for a proportion measures how many standard errors an observed rate sits from the baseline:

$$z = \frac{\hat{p} - p_0}{\sqrt{p_0(1 - p_0)/n}}$$

A flag only fires when *both* the effect size (e.g. bounce $> 1.3\times$ average) *and* significance ($|z| \geq 1.645$, $\approx 95\%$ confidence) hold. Flags on the same page are annotated as co-occurring so the AI treats them as one causal story, not N independent findings.

4.5 Service 3 — AI Service (ai service)

Stack: Node.js + Express, multi-provider LLM layer (Groq, Gemini, Mistral, Cerebras, Ollama).

- **Provider-agnostic core.** A single `llm.js` entry point resolves the provider from the `AI_PROVIDER` env var and lazily loads only the SDK actually selected, so a missing optional dependency never breaks the service. All clients share a uniform `{ content, meta }` shape.
- **Generate→Judge orchestration.** When `AI_PROVIDER=orchestrate`, several generator models draft candidates in parallel; candidates are deduplicated by fingerprint (`category:page`, tracking cross-model agreement); a judge model merges, filters, and re-ranks them. Any generator that is rate-limited is simply skipped (`Promise.allSettled`); if the judge fails, the deduped candidates are returned so a report always comes back.
- **Vision-integrated path.** When the crawler produced screenshots, a vision model (Gemini) reasons over the images directly in the same pass that writes suggestions — a visual-flaw pre-pass also injects `dataQuality:"visual"` flags so UI problems become first-class inputs alongside the numbers.
- **Deterministic guardrails.** Every raw LLM item passes through `validateItems`: enums and page paths are repaired, cited numbers are grounded against the real data (ungrounded claims downgraded or dropped when `GROUNDING_STRICT=true`), and duplicates are folded. `forecastImpact` then attaches a *deterministic*, data-derived revenue estimate — never the LLM’s own guess.
- **On-demand code generation.** `/code/generate` pulls the real theme files for the affected page from `shopify-pp`, caps total theme content at 150K chars (OOM defense), and asks a code-strong model (Gemini by default) for a *surgical* patch: only the changed lines, plus verbatim anchor text so a developer — or a script — can locate the exact insertion point.

4.6 Service 4 — Review Hub (review-hub)

Stack: React + Vite, Recharts, `localStorage` persistence.

- **Two personas.** PO/admin sees the full dashboard and the approve/reject controls; developers see only approved suggestions *with* their code patches.

- **Resilient by design.** If the backend services are down, the dashboard falls back to an embedded demo dataset, so it is always demoable. The active data source (`ai` / `mock` / `demo`) is shown in the header.
- **Zero-backend state.** Review status persists in `localStorage`; the app talks directly to the Analytics and AI services from the browser.

5 Product Management & Delivery

Beyond the engineering, the project was run as a structured product engagement — the artifacts below are as much a part of the deliverable as the code.

5.1 Agile backlog

The backlog is a three-tier Agile hierarchy (**Epics** → **Features** → **User Stories**), using **MoSCoW** prioritization and **Given–When–Then** (GWT) acceptance criteria throughout. It is structured across four epics:

Epic	Description	Status	Scope
E-001	Data ingestion & normalization	Complete	MVP (v1)
E-002	Analytics Service REST API (Node/Express)	Complete	MVP (v1)
E-003	AI Suggestion Service — LLM recommendation generation	Complete	MVP (v1)
E-004	Code generation / developer-facing patch output	Complete	MVP (v1)

All four epics are complete and in MVP scope. E-004 (developer-facing code generation) is realized as the AI service's `/code/generate` endpoint, which fetches the real theme code for the affected page from the extractor and generates a surgical, ready-to-paste patch.

5.2 Key product decisions (and their rationale)

- **Two LLM roles, deliberately split.** Fast, real-time *suggestion* generation vs. slower, code-strong *patch* generation are different jobs and use different models — this split should not be collapsed into “just use one model”. (Handover pinned Groq Llama 3.3 70B for suggestions and Gemini for code; the current code generalizes suggestions to a provider-agnostic layer with optional multi-model orchestration, and keeps Gemini as the default code model.)
- **Confidence capping over fabrication.** Some Shopify metrics (bounce rate, cart-abandonment rate, average load time) can return `null`. Rather than estimate or fabricate a value, the system *caps* the confidence of any recommendation touching those metrics. This is a principled choice to avoid hallucinated numbers — framed to merchants as a *managed limitation*, not an unresolved gap.
- **Detectors skip silently when data is missing.** The optional detectors (`cta_below_fold`, `poor_quality_traffic_source`) depend on crawler data that only exists when a website URL is supplied. With no URL they skip rather than guess — no hallucinated output when the underlying data is absent.
- **Approval as the hand-off trigger.** A PO approving a recommendation is precisely what promotes it into the developer view as implementation-ready — the mechanism that keeps a human between the AI and the storefront.

- **Mock data is always labelled.** Demo/unsynced accounts see mock data clearly marked as test-store data, so it is never confusable with a real merchant's live analytics.

5.3 Competitive & discovery context

Product discovery benchmarked the engine against experience-analytics and CRO tools (*Contentsquare*, *VWO*, *AICRO.io*), AI coding assistants (*Cursor*) as a reference for the code-generation experience, and comparable consultancy players (*Accenture*, *Infosys*, *EPAM*, *Thoughtworks*). The differentiator that emerged: not another analytics dashboard, but the *translation layer* from measurement to a reviewed, implementable code change.

6 How to Connect All Services

The services are wired together entirely through URLs in environment variables and one frontend config file. Getting these right is the whole integration.

6.1 The wiring map

Consumer	Setting	Points at
shopify-pp	ANALYTICS_SERVICE_URL (env)	Analytics /api/analytics/ingest
ai service	ANALYTICS_SERVICE_URL (env)	Analytics base URL
ai service	SHOPIFY_APP_URL (env)	Extractor base URL (for theme code)
review-hub	ANALYTICS_URL (config.js)	Analytics base URL
review-hub	AI_URL (config.js)	AI service base URL
review-hub	SHOPIFY_APP_URL (config.js)	Extractor base URL

6.2 Local development

Install dependencies once per service, then start each in its own terminal in dependency order (Analytics first, since AI depends on it):

```
# 1. Analytics Service -> http://localhost:4000 (no env vars needed)
cd "analytic service" && npm install && npm start

# 2. AI Service -> http://localhost:5001
cd "ai service" && npm install
cp .env.example .env # add GROQ / MISTRAL / CEREBRAS / GEMINI keys
npm start

# 3. Shopify Extractor -> http://localhost:3000
cd shopify-pp && npm install
cp .env.example .env # add Shopify client id/secret, ENCRYPTION_KEY
npm start

# 4. Review Hub -> http://localhost:5173
cd review-hub && npm install && npm run dev
```

6.3 Exposing local services with Cloudflare quick tunnels

Because the Review Hub runs in the *browser*, it cannot reach a teammate's `localhost`. A Cloudflare quick tunnel mints a public HTTPS URL for a local port with no account or deploy required — useful to dodge free-tier rate limits on hosted LLM backends:

```
brew install cloudflared
cloudflared tunnel --url http://localhost:4000 # -> Analytics tunnel URL
cloudflared tunnel --url http://localhost:5001 # -> AI tunnel URL
```

Paste the printed `*.trycloudflare.com` URLs into `review-hub/src/lib/config.js` (`ANALYTICS_URL`, `AI_URL`) and `ai service/.env` (`ANALYTICS_SERVICE_URL`).

Two gotchas that cost real debugging time

(1) **Tunnel URLs are single-use** — each restart mints a brand-new random URL; you must re-paste it everywhere. (2) **dotenv** only reads `.env` once at startup, so **restart the AI service after editing its `.env`**. Vite's dev server, by contrast, hot-reloads `config.js` on save.

7 Deployment — Vercel, Railway, Cloudflare & More

Each service has different runtime needs, so no single platform is optimal for all four. The table below maps each service to its best-fit host and why.

Service	Recommended host	Why
Review Hub	Vercel / Cloudflare Pages	Static Vite build; global CDN; zero server
Shopify Extractor	Railway / Fly.io	Long-lived Node process + persistent SQLite volume + webhooks
Analytics Service	Railway / Fly.io (Docker)	Needs Playwright/Chromium; ships its own Dockerfile
AI Service	Railway / Fly.io	Long-lived Node process; outbound LLM calls; env secrets

7.1 Review Hub on Vercel (or Cloudflare Pages)

The Review Hub is a pure static build — ideal for a CDN host.

```
# From the review-hub/ directory
npm install -g vercel
vercel # first run links the project
vercel --prod # deploys the production build
```

- **Build command:** `npm run build` **Output dir:** `dist`
- Because service URLs currently live in `src/lib/config.js` (not env vars), point them at your *deployed* Analytics/AI URLs before building. *Recommended improvement:* read them from `import.meta.env.VITE_*` so the same build works across environments (see Section 8).
- **Cloudflare Pages** is an equivalent alternative: connect the repo, set build command `npm run build`, output `dist`.

7.2 Shopify Extractor & AI Service on Railway

Railway builds Node apps with Nixpacks automatically and provides persistent volumes — essential for the extractor's SQLite database.

```
npm install -g @railway/cli
railway login
railway init # run inside each service folder
railway up # build & deploy
```

- **Start command:** `npm start` for both.
- **Extractor env vars:** `SHOPIFY_CLIENT_ID`, `SHOPIFY_CLIENT_SECRET`, `SHOPIFY_SCOPES`, `APP_URL` (the public Railway URL), `ANALYTICS_SERVICE_URL`, `ENCRYPTION_KEY`, and `DB_PATH` pointing at a **mounted volume** (e.g. `/data/shops.db`) — otherwise every redeploy wipes installed shops.
- **AI env vars:** the provider API keys, `AI_PROVIDER`, `ANALYTICS_SERVICE_URL`, `SHOPIFY_APP_URL`.
- **Update the Shopify app config:** `shopify.app.toml`'s `application_url` and `redirect_urls` must match the deployed extractor URL.

7.3 Analytics Service via Docker (Railway / Fly.io / any container host)

The Analytics Service needs Chromium, so it ships a Dockerfile built on the official Playwright image:

```
FROM mcr.microsoft.com/playwright:v1.61.1-jammy
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY . .
CMD ["node", "index.js"]
```

```
# Railway auto-detects the Dockerfile; or on Fly.io:
fly launch --dockerfile Dockerfile
fly deploy
```

- Optionally set AWS S3 credentials so screenshots persist in object storage instead of the container's ephemeral filesystem.
- Ensure the host allocates enough memory — Chromium is memory-hungry; Railway's smallest tier can OOM on image-heavy crawls.

7.4 Cloudflare's role

Cloudflare serves two distinct purposes here, which are easy to confuse:

1. **Quick tunnels** (`cloudflared tunnel -url`) — an ephemeral dev bridge from a public URL to a local port (Section 6). Not for production.
2. **Cloudflare Pages** — a production static host for the Review Hub, equivalent to Vercel.

Note that **Playwright-based crawling and long-lived Express servers cannot run on Cloudflare Workers** (no full Node/Chromium runtime), which is why the backend services target Railway/Fly rather than Cloudflare's edge.

7.5 Deployment checklist

1. Deploy Analytics (Docker) → note its URL.
2. Deploy AI Service with `ANALYTICS_SERVICE_URL` set → note its URL.
3. Deploy Extractor with `ANALYTICS_SERVICE_URL` + `APP_URL` + persistent `DB_PATH`.
4. Update `shopify.app.toml` URLs to the deployed extractor.
5. Build & deploy Review Hub with all three URLs configured.
6. Smoke-test the pipeline using `TESTING.md`'s end-to-end order.

8 How Can We Improve It

The current system is a strong, demoable prototype. The improvements below are grouped by theme and roughly ordered by leverage.

8.1 Persistence & state

- **Replace in-memory caches with a real datastore.** Both the Analytics cache and the AI service's report/suggestion state are in-memory objects — a restart wipes everything. Move to Postgres or Redis so reports, review decisions, and suggestion status survive redeloys and scale beyond one process.
- **Move review state off `localStorage`.** The Review Hub's approve/reject decisions live only in the browser; they should live server-side so decisions are shared across users and devices and auditable.

8.2 Configuration & security

- **Environment-driven frontend config.** Hard-coded URLs in `config.js` require a rebuild per environment; switch to `VITE_*` env vars so one artifact deploys everywhere.
- **Real authentication.** The Review Hub uses hard-coded users. Replace with proper auth (Shopify session tokens, or an SSO provider) and role-based access control.
- **Lock down CORS.** Services currently allow all origins; scope them to known frontends in production.

8.3 Reliability & observability

- **Async job queue for crawling & generation.** Crawls and LLM calls are slow and rate-limited; a queue (BullMQ / a managed queue) with retries and backpressure beats fire-and-forget async.

- **Structured logging, metrics, and tracing.** Today's insight is `console.log`; add request tracing across the four services so a failed report can be diagnosed end-to-end.
- **Cost & quota telemetry.** Token usage is already tracked in `meta`; surface it as a dashboard so provider costs and rate-limit headroom are visible before they cause 503s.

8.4 AI quality

- **Close the feedback loop.** Feed PO approve/reject decisions back as training signal (few-shot examples or fine-tuning) so suggestion quality improves over time per merchant.
- **Patch verification before it reaches a developer.** Lint or dry-apply the generated patch against the fetched theme files, so a developer never receives a patch whose anchor text no longer matches.
- **Regression suite for detectors and prompts.** The fixtures directory is a good start; grow it into a golden-dataset test so prompt or threshold changes are measured, not guessed.

9 Future Scope

9.1 Immediate next steps (open items)

With MVP scope complete, the open items are next-phase work rather than unfinished MVP tasks. In priority order:

1. **Run the first merchant pilot.** The single most important next step — v1 is only validated once one recommendation is implemented on a real store and its impact measured. Everything else is subordinate to this.
2. **Harden code generation from the pilot.** E-004 is implemented; the pilot is what proves the PO-approval→patch flow on real theme code and surfaces the rough edges.
3. **Add data depth (GA4 and beyond)** to reduce reliance on confidence capping — the first concrete step toward the universal-connector vision below.
4. **Specify rate limits concretely.** “Generous limits” was flagged during backlog review as under-specified; real numbers are needed before any external-facing rollout.
5. **Frame crawler optionality correctly.** When explaining why the two optional detectors need a website URL, frame it as a barrier-to-entry and a time-vs-quality tradeoff for the merchant — not around token limits.

9.2 Vision

Today the pipeline is Shopify-specific and ends by handing a developer a patch to copy-paste. The future vision inverts both limitations:

Where this is going

A library of lightweight **extensions** / **connectors** pulls *raw data* from any analytics platform. That data is turned into ranked suggestions and generated code *exactly as today*. But instead of a human copy-pasting the patch, the approved change is **applied to the codebase automatically** — committed, opened as a pull request, and merged — with no manual step. And it is **no longer limited to Shopify**.

9.3 Pillar 1 — Universal connectors (beyond Shopify)

The extractor becomes one of many pluggable *connectors* behind a common interface. Each connector’s only job: authenticate to a platform and emit raw data in the shape the Analytics Service already expects (**overview**, **pages**, **trafficSources**, **devices**, **funnel**, Core Web Vitals). Because the ingest contract is already platform-neutral, everything downstream — detectors, AI, review — works unchanged. Candidate connectors:

Connector	Raw data it would emit
Google Analytics 4	Sessions, conversions, funnels, traffic sources, devices
Adobe Analytics	Enterprise session & funnel data
WooCommerce / Magento / BigCommerce	Non-Shopify e-commerce metrics + theme/template code
Plausible / Matomo	Privacy-first web analytics
Mixpanel / Amplitude	Product-analytics event funnels
Search Console / PageSpeed	SEO & Core Web Vitals at scale

The connector interface is deliberately thin: `authenticate()`, `extract()` → normalized payload, and optionally `getSourceCode(page)` for platforms where the code is fetchable. A connector that cannot supply source code simply skips the auto-apply pillar and still produces suggestions.

9.4 Pillar 2 — Automatic code application

The single biggest manual step today is a developer copy-pasting the patch. The patch format was *designed* to remove that step: each patch already carries verbatim anchor text (“insert after this exact line ...”) precisely so a *script* — not just a human — can locate the edit. The roadmap:

1. **Anchor-based patch applier.** A deterministic engine locates the anchor by exact string match and applies the change, failing loudly if the anchor drifted (never a fuzzy guess on live code).
2. **Git integration.** For code-backed platforms, the applier commits to a branch and opens a **pull request** — the PO’s “approve” becomes the trigger, and the human review gate moves to standard PR review.
3. **Shopify theme write-back.** Using the Shopify Theme API, apply the change to a *duplicated draft theme* first, screenshot the result, and only publish after approval.
4. **Verify-before-merge.** Re-run the crawler on the changed page and confirm the flagged problem is actually resolved before the PR is allowed to merge — closing the observe→act→verify loop automatically.

9.5 Pillar 3 — Continuous optimization

With connectors pulling data on a schedule and patches applying themselves, the system becomes a *continuous* optimization engine rather than a one-shot report: detect → suggest → (PO-approve) → apply → measure → repeat. A/B testing each applied change against a holdout would let the system *prove* lift rather than forecast it, and feed real outcomes back into suggestion ranking.

9.6 Target architecture

Layer	Evolution
Connectors	Shopify extractor → pluggable connector SDK (GA4, Woo, Adobe, ...)
Analytics	Unchanged contract; add scheduled/continuous ingestion
AI	Unchanged generation; add feedback-trained ranking
Apply	<i>New</i> : anchor-based applier + Git/Theme write-back + verify
Review	PO approval becomes the PR/apply trigger, not a copy-paste handoff

The through-line

Every future pillar reuses the platform-neutral ingest contract and the anchor-carrying patch format that already exist. The prototype was, in effect, built as the Shopify *instance* of a general system — the roadmap is mostly about widening the front door (connectors) and automating the last step (apply), not rewriting the middle.

10 Conclusion

Exp Intelligence demonstrates that the valuable, hard part of storefront optimization is not measurement but the *translation from measurement to implementation*. By chaining statistical rigor, multimodal (numbers + visual) reasoning, and theme-grounded code generation — all behind a human review gate — it turns a wall of analytics into a short, ranked list of actions with the code to ship them. The architecture's platform-neutral ingest contract and script-locatable patch format mean the ambitious future — universal connectors and automatic code application beyond Shopify — is an extension of what exists, not a rebuild.

End of report